Table des matières

Cours 07 — Charger en incrémental 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — La caisse, livrée jour après jour · Fiabiliser et faire tourner 2
Étape 2 — Le fait incrémental, et le filtre de tout le monde · MARTS 3
Étape 3 — Ce que le 13 juin a perdu · Fiabiliser et faire tourner 6
Étape 4 — L'instant de l'export, de la RAW au fait · Fiabiliser et faire tourner 7
Étape 5 — Le mois entier, et une passe relancée · Fiabiliser et faire tourner 10
Étape 6 — Versionner · Fiabiliser et faire tourner 12
À retenir 12

Cours 07 — Charger en incrémental

Cours 7/9 · La Brûlerie — Formation Analytics Engineering

STAGING	INTERMEDIATE	MARTS
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
✓ ici · Fiabiliser et faire tourner, ce qui traverse les trois couches : les tests, l'incrémental, l'intégration continue	✓	✓

Ce que vous allez faire

Jusqu'ici, la `RAW` arrivait d'un bloc, et `dbt build` refaisait le fait des ventes en entier. En exploitation, la caisse exporte chaque soir, et le chargement de nuit ajoute cet export à la `RAW`. Refaire le fait chaque nuit, c'est recalculer tout l'historique pour y ajouter une journée ; un fait incrémental ne traite que ce qui est arrivé depuis la passe précédente. Ce cours fait arriver juin jour après jour et rend `fct_ventes` incrémental. Le filtre que tout le monde écrit d'abord y perd la fin du 12 juin ; le cours écrit ensuite celui qui tient, et prouve qu'une passe relancée ne change rien.

la caisse livrée jour après jour → `fct_ventes` incrémental, filtré sur le jour de vente → ce que le 13 juin perd → l'instant de l'export, de la `RAW` au fait → le mois entier, et une passe relancée

© 2026 iAlexMG. Tous droits réservés.

Les fichiers à écrire se donnent en pseudocode, et le code complet est dans `Codes_07_AE.md`, à ouvrir après avoir essayé.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, la livraison de la caisse, dbt et Git ; la seconde pour le notebook de DuckDB, qui s'ouvre à l'étape 1 et reste ouvert jusqu'à l'étape 5. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
set DBT_PROJECT_DIR=02_ae
REM Attendu : rien ne s'affiche.
dbt build
REM Attendu : 43 of 43 : treize modèles, le snapshot et 29 tests, puis Done. PASS=43 WARN=0 ERROR=0.
```

Ce cours part du projet tel que le cours 06 l'a laissé. S'il manque un test ou un fait, reprenez la fin du cours 06 avant de continuer.

Étape 1 — La caisse, livrée jour après jour · Fiabiliser et faire tourner

Chaque soir, sept minutes après la fermeture, la caisse exporte les tickets du jour et leurs lignes ; la nuit, le chargeur de Data Engineering, `01_de\charger.py`, ajoute ces deux fichiers à la `RAW`. Il prend un jour en argument, et ne charge que les exports faits jusqu'à ce jour-là et que son journal ne connaît pas encore. L'instant d'un export se lit à la fin du nom de son fichier, `AAAAMMJJHHMM`, comme le dit le dictionnaire des sources, section 1.

Le cours 00 a rempli la `RAW` d'un bloc, le mois entier. Pour la voir arriver comme en exploitation, le paquet de départ fournit `ramener_caisse.py`, qui remonte le temps : il retire de `raw_tickets` et de `raw_lignes` les exports faits après le jour qu'on lui donne, et retire leurs fichiers du journal de Data Engineering. Le chargeur livre ensuite les jours suivants, un à la fois. Les cinq autres tables de la `RAW` ne bougent pas, et livrée jusqu'au 30 juin, la caisse redevient celle du cours 00, ligne pour ligne.

Le notebook, les scripts et dbt se partagent l'entrepôt. Or DuckDB ne laisse écrire dans un fichier que le programme qui est seul à l'ouvrir : tant que le notebook tient l'entrepôt, même en lecture seule, les scripts et dbt sont refusés. Le notebook attache donc l'entrepôt pour le lire, et le détache avant chaque livraison et chaque commande dbt.

1.1 Ramener la caisse au 11 juin, dans la première fenêtre.

```
python ..\02_Analytics_Engineering\01_Brulerie_facile\Paquet_de_depart\ramener_caisse.py --au 2025-06-11
REM Attendu : raw_tickets, 11 exports et 575 lignes ; raw_lignes, 11 exports et 959 lignes ; le
REM dernier export le 2025-06-11 à 18:07:00 ; puis Ramené : la caisse porte ses exports jusqu'au 2025-06-11.
```


© 2026 iAlexMG. Tous droits réservés.

Si le script s'arrête sur	La cause, et le geste
`brulerie.duckdb est ouvert ailleurs, sans doute par le notebook`	le notebook tient l'entrepôt : relancer sa seconde cellule, le `detach`, puis la commande
`Aucun entrepôt dans …`	la `RAW` n'est pas préparée : faire le cours 00

1.2 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « AE Cours 07 ». Laisser cette fenêtre ouverte : la fermer arrête le notebook.

1.3 Écrire les deux premières cellules du notebook, qui resserviront tout le cours. La première attache l'entrepôt en lecture seule ; l'exécuter.

```
attach if not exists 'plateforme/entrepot/brulerie.duckdb' as brulerie (read_only);
-- Attendu : brulerie paraît sous Attached databases.
```

La seconde le détache ; l'écrire sans l'exécuter. Grâce à `if not exists` et à `if exists`, l'une et l'autre se relancent sans erreur, même deux fois de suite.

```
detach database if exists brulerie;
```

Chaque requête qui suit va dans une cellule neuve, sous ces deux-là.

1.4 Compter les tickets de la vue.

```
select count(*) as tickets, max(horodatage) as dernier_ticket
from brulerie.stg_tickets;
-- Attendu : 575 tickets, le dernier le 2025-06-11 à 17:46:50.
```

1.5 Compter les lignes du fait.

```
select count(*) as lignes, max(cle_date) as dernier_jour
from brulerie.fct_ventes;
-- Attendu : 2509 lignes, le dernier jour le 2025-06-30.
```

La vue relit la `RAW` à chaque requête, et elle voit le 11 juin. Le fait est une table : dbt l'a calculé à votre dernier `dbt build`, et il porte encore tout juin. Une table ne bouge qu'au passage de dbt.

Étape 2 — Le fait incrémental, et le filtre de tout le monde · MARTS

Un modèle incrémental est une table que dbt ne refait pas. À chaque passe, il lance la requête du modèle sur ce qui est arrivé seulement, et range le résultat dans la table qui existe déjà. Trois réglages le décrivent :

© 2026 iAlexMG. Tous droits réservés.

Le réglage	Ce qu'il dit
`materialized: incremental`	une table, complétée à chaque passe au lieu d'être refaite
`unique_key: [ticket_id, ligne]`	ce qui désigne la même ligne d'une passe à l'autre : le grain du fait
`incremental_strategy: delete+insert`	une ligne de la passe dont la clé est déjà dans la table remplace l'ancienne : dbt efface, puis insère

Les réglages d'un modèle s'écrivent dans son fichier SQL, ou dans le YAML qui le décrit. Ceux-ci vont dans `_marts.yml`, à côté des tests du fait, et le cours 08 y ajoutera son contrat. Ils l'emportent sur le `+materialized: table` que `dbt_project.yml` donne aux `MARTS`.

Reste le filtre, qui dit ce qu'une passe prend. Dans la requête, `{% if is_incremental() %}` ouvre un bloc que dbt ne garde que pour une passe incrémentale : quand la table existe déjà, et qu'on ne lui demande pas de la refaire. `{{ this }}` désigne la table elle-même, telle que les passes précédentes l'ont laissée. Le filtre que tout le monde écrit d'abord ne prend que les jours de vente plus récents que le dernier jour déjà dans le fait. Ce dernier jour se lit par une sous-requête scalaire, une requête entre parenthèses qui rend une seule valeur : elle s'apprend au Cours 07 de la formation SQL avancé, `Theorie_07_SQL.md`, section 2.

2.1 Régler le fait, dans la première fenêtre.

```
notepad 02_ae\models\marts\_marts.yml
```

Sous `fct_ventes`, le commentaire 3 s'allonge, et un bloc `config:` s'ajoute entre la description et les colonnes. Le reste du fichier ne bouge pas :

```
  # 3. le fait, bâti par passes : chaque clé recalculée existe dans sa dimension
  - name: fct_ventes
    description: "Le fait des ventes, une ligne par ligne d'un ticket payé."
    config:
      materialized: ____
      incremental_strategy: delete+insert
      unique_key: [____, ____]
    columns:
```

Enregistrer, puis fermer le Bloc-notes.

2.2 Écrire le filtre.

```
notepad 02_ae\models\marts\fct_ventes.sql
```

Le fichier garde ses trois parties, et gagne à la fin le commentaire 4 et le bloc du filtre :

```
-- 02_ae/models/marts/fct_ventes.sql : le fait des ventes, une ligne par ligne d'un ticket payé
select
    -- 1. une clé vers chaque dimension, calculée comme dans la dimension
```


© 2026 iAlexMG. Tous droits réservés.

```
    date_vente as cle_date,
    md5(produit_id::varchar) as cle_produit,
    md5(membre_id) as cle_membre,
    -- 2. le ticket et sa ligne, gardés dans le fait
    ticket_id,
    ligne,
    -- 3. les mesures
    quantite,
    quantite * prix_unitaire as montant_brut,
    rabais,
    montant
from {{ ref('int_ventes') }}
-- 4. une passe ne garde que ce qui est arrivé depuis la précédente : les jours de vente plus récents
{% if ____() %}
where date_vente > (select max(____) from {{ ____ }})
{% endif %}
```

Enregistrer, puis fermer le Bloc-notes.

2.3 Refaire le fait sur les onze jours livrés. La table existe déjà : dbt passerait tout de suite en régime incrémental, et garderait le mois entier, que le filtre ne regarde pas. `--full-refresh` refait la table de zéro, par la requête entière, sans le filtre. Relancer la seconde cellule du notebook, le `detach`, puis dans la première fenêtre :

```
dbt run --select fct_ventes --full-refresh
REM Attendu : 1 of 1 OK created sql incremental model main.fct_ventes, puis Done. PASS=1.
```

Relancer la première cellule du notebook, l'`attach`, puis la requête de l'étape 1.5.

```
select count(*) as lignes, max(cle_date) as dernier_jour
from brulerie.fct_ventes;
-- Attendu : 949 lignes, le dernier jour le 2025-06-11.
```

2.4 Livrer le 12 juin, puis faire passer le fait. Chaque livraison se fait en trois temps :

Temps	Où	Quoi
1	le notebook	relancer la seconde cellule, le `detach` : le notebook lâche l'entrepôt
2	la première fenêtre	livrer la caisse par le chargeur de Data Engineering, puis `dbt run`

© 2026 iAlexMG. Tous droits réservés.

Temps	Où	Quoi
3	le notebook	relancer la première cellule, l'`attach` : le notebook relit l'entrepôt

```
python 01_de\charger.py 2025-06-12
REM Attendu : rien ne s'affiche ; le chargeur a pris l'export du 12 à 15 h 42, ses tickets et ses
REM lignes : la caisse en porte 12 exports, 633 tickets et 1053 lignes.
dbt run --select fct_ventes
REM Attendu : 1 of 1 OK created sql incremental model main.fct_ventes, puis Done. PASS=1.
```

Le 12 juin, la caisse a exporté la journée à la main, à 15 h 42 : le dernier export du jour est celui-là. Lire les derniers jours du fait, dans une cellule neuve :

```
-- 1. les derniers jours du fait : les tickets et les lignes de chacun
select cle_date, count(distinct ticket_id) as tickets, count(*) as lignes
from brulerie.fct_ventes
where cle_date >= '2025-06-10'
group by cle_date
order by cle_date;
-- Attendu : 2025-06-10, 32 tickets et 57 lignes ; 2025-06-11, 47 et 77 ; 2025-06-12, 58 et 94.
```

La passe a ajouté le 12 juin, et rien d'autre : les 58 tickets faits avant l'export de 15 h 42, le dernier à 15:03:44.

Étape 3 — Ce que le 13 juin a perdu · Fiabiliser et faire tourner

Le 13 juin, à 7 h 05, la caisse a repris l'export du 12 : 65 tickets, contre 58 la veille. Le soir, elle a exporté le 13. La nuit suivante, les deux exports arrivent ensemble.

3.1 Livrer le 13 juin, puis faire passer le fait, en trois temps (`detach`, livraison et passe, `attach`).

```
python 01_de\charger.py 2025-06-13
REM Attendu : rien ne s'affiche ; la caisse porte maintenant 14 exports, 752 tickets et 1249 lignes,
REM le dernier le 2025-06-13 à 18:07:00.
dbt run --select fct_ventes
REM Attendu : 1 of 1 OK created sql incremental model main.fct_ventes, puis Done. PASS=1.
```

3.2 Relancer la cellule des derniers jours, celle de l'étape 2.4. Attendu : 2025-06-10, 32 tickets et 57 lignes ; 2025-06-11, 47 et 77 ; 2025-06-12, 58 et 94 ; 2025-06-13, 54 et 90.

Le 13 juin est entré. Le 12 a toujours 58 tickets, alors que son second export en porte 65.

© 2026 iAlexMG. Tous droits réservés.

3.3 Chercher ce qui manque. La vue `int_ventes` relit toute la `RAW` livrée ; le fait n'a que ce que ses passes ont pris. Comparer les deux est le contrôle qui coûte le moins. La jointure externe à gauche garde chaque ligne de la vue ; une ligne que le fait n'a pas y garde vides les colonnes du fait.

```
-- 1. les lignes vendues que le fait n'a pas : dans la vue, absentes du fait
select v.date_vente, count(distinct v.ticket_id) as tickets, count(*) as lignes,
       sum(v.montant) as montant
from brulerie.int_ventes as v
left join brulerie.fct_ventes as f on f.ticket_id = v.ticket_id and f.ligne = v.ligne
where f.____ is null
group by v.date_vente;
-- Attendu : 2025-06-12, 6 tickets, 8 lignes, 55.50.
```

Sept tickets ont été faits après l'export de 15 h 42, de 15 h 45 à 17 h 16 ; six sont payés, le septième annulé. Ils sont arrivés avec l'export de 7 h 05, mais leur jour de vente est le 12, et le fait avait déjà le 12 : le filtre les a écartés, sans erreur. Aucune passe ne les reprendra, puisque le dernier jour du fait ne fait que monter.

3.4 Interroger le témoin. Le cours 05 l'a écrit pour cela : il compare le fait à la caisse, ticket par ticket. Relancer le `detach` dans le notebook, puis dans la première fenêtre :

```
dbt test --select assert_temoin_caisse
REM Attendu : 1 of 1 FAIL 6 assert_temoin_caisse, Got 6 results, configured to fail if != 0, puis ERROR=1.
```

Les six tickets payés de la fin du 12 juin. Rien d'autre ne le dit : le grain tient, chaque clé trouve sa dimension, et le total du mois se lirait comme un total juste. Le filtre incrémental n'a pas de filet ; seule une comparaison avec la source voit ce qu'il perd.

Étape 4 — L'instant de l'export, de la `RAW` au fait · Fiabiliser et faire tourner

Le jour de vente dit quand le ticket a été fait, pas quand il est arrivé. Une passe doit prendre ce qui est arrivé depuis la précédente : chaque ligne d'un export plus récent que le dernier que le fait a vu. L'instant de l'export est dans la `RAW`, à la fin du nom du fichier ; il monte jusqu'au fait, qui en garde la trace, et le filtre le compare au plus récent déjà rangé. Le 13 juin, l'export de 7 h 05 est plus récent que celui de 15 h 42 : ses 65 tickets passent, les 58 déjà rangés comme les sept nouveaux.

Les 58 ne doublent pas. Leur clé, le ticket et sa ligne, est déjà dans le fait, et `delete+insert` efface l'ancienne ligne avant d'insérer la nouvelle. Sans `unique_key`, dbt ajouterait les 102 lignes payées du nouvel export aux 94 déjà là : 196 lignes le 12 juin, pour 1 299,50 $, et le test du grain en faute sur 94 couples. Une ligne reçue de nouveau remplace l'ancienne, sans la doubler.

Pour tirer l'instant	**L'outil**
le dernier morceau du nom, après le deuxième _	`split_part(filename, '_', 3)`, qui rend `202506130705.csv`
sans l'extension	`replace(…, '.csv', '')`

© 2026 iAlexMG. Tous droits réservés.

Pour tirer l'instant	L'outil
en horodatage	`strptime(…, '%Y%m%d%H%M')` : l'année, le mois, le jour, l'heure et la minute

Trois fichiers changent, un par couche : la vue des lignes tire l'instant, `int_ventes` le laisse passer, le fait le garde et le filtre dessus.

4.1 Tirer l'instant de l'export, dans la première fenêtre.

```
notepad 02_ae\models\staging\stg_lignes.sql
```

La ligne de `montant` prend une virgule, le nouveau commentaire 2 et sa ligne la suivent, et les anciens commentaires 2 et 3 deviennent 3 et 4 :

```
-- 02_ae/models/staging/stg_lignes.sql : les lignes des tickets, typées, une par ligne de ticket
select
    -- 1. typer : le texte de la RAW devient des nombres, sous le même nom
    ticket_id::integer as ticket_id,
    ligne::integer as ligne,
    produit_id::integer as produit_id,
    description,
    quantite::integer as quantite,
    prix_unitaire::decimal(10, 2) as prix_unitaire,
    rabais::decimal(10, 2) as rabais,
    code_rabais,
    code_taxe,
    montant::decimal(10, 2) as montant,
    -- 2. l'instant de l'export, à la fin du nom du fichier : AAAAMMJJHHMM
    strptime(replace(split_part(filename, '_', ____), '.csv', ''), '____') as exporte_le
-- 3. lire la RAW par sa source
from {{ source('brulerie', 'raw_lignes') }}
-- 4. tirer l'état courant : de chaque ligne de ticket, garder l'export le plus récent
qualify row_number() over (partition by ticket_id, ligne order by filename desc) = 1
```

Enregistrer, puis fermer le Bloc-notes.

4.2 Le laisser passer par `int_ventes`.

```
notepad 02_ae\models\intermediate\int_ventes.sql
```

Le commentaire 1 s'allonge, et l'instant s'ajoute après `l.montant`, qui prend une virgule :

```
-- 02_ae/models/intermediate/int_ventes.sql : les lignes des tickets payés, avec leur date et leur membre
select
    -- 1. la ligne, avec la date et le membre de son ticket, et l'instant de son export
    l.ticket_id,
```


© 2026 iAlexMG. Tous droits réservés.

```
    l.ligne,
    t.horodatage::date as date_vente,
    t.membre_id,
    l.produit_id,
    l.quantite,
    l.prix_unitaire,
    l.rabais,
    l.montant,
    l.____
-- 2. les lignes et leur ticket, lus par ref()
from {{ ref('stg_lignes') }} as l
inner join {{ ref('stg_tickets') }} as t on t.ticket_id = l.ticket_id
-- 3. ce qui est une vente : un ticket payé ; les annulés sortent, le remboursement reste
where t.statut = 'PAYE'
```

Enregistrer, puis fermer le Bloc-notes.

4.3 Le garder dans le fait, et filtrer dessus.

```
notepad 02_ae\models\marts\fct_ventes.sql
```

L'instant rejoint le ticket et sa ligne, au commentaire 2, et le filtre change de colonne :

```
-- 02_ae/models/marts/fct_ventes.sql : le fait des ventes, une ligne par ligne d'un ticket payé
select
    -- 1. une clé vers chaque dimension, calculée comme dans la dimension
    date_vente as cle_date,
    md5(produit_id::varchar) as cle_produit,
    md5(membre_id) as cle_membre,
    -- 2. le ticket et sa ligne, gardés dans le fait, et l'instant de l'export qui l'a livrée
    ticket_id,
    ligne,
    ____,
    -- 3. les mesures
    quantite,
    quantite * prix_unitaire as montant_brut,
    rabais,
    montant
from {{ ref('int_ventes') }}
-- 4. une passe ne garde que les lignes d'un export plus récent que le dernier déjà rangé
{% if is_incremental() %}
where ____ > (select max(____) from {{ this }})
{% endif %}
```


© 2026 iAlexMG. Tous droits réservés.

Enregistrer, puis fermer le Bloc-notes.

4.4 Rejouer le 12 juin. Le filtre a changé, et les passes de l'étape 3 ont laissé le 12 incomplet : le fait se refait de zéro. `+fct_ventes`, un `+` devant le nom, prend aussi tout ce que le modèle lit, dont les deux vues qui viennent de changer. La caisse se ramène au 11 juin, puis le chargeur livre le 12. En trois temps (`detach`, livraison et passe, `attach`) :

```
python ..\02_Analytics_Engineering\01_Brulerie_facile\Paquet_de_depart\ramener_caisse.py --au 2025-06-11
REM Attendu : le dernier export le 2025-06-11 à 18:07:00.
python 01_de\charger.py 2025-06-12
REM Attendu : rien ne s'affiche.
dbt run --select +fct_ventes --full-refresh
REM Attendu : 4 of 4, les vues stg_lignes, stg_tickets et int_ventes, puis le fait, et PASS=4.
```

Dans une cellule neuve :

```
-- 1. les derniers jours du fait, et l'instant du plus récent export de chacun
select cle_date, count(distinct ticket_id) as tickets, count(*) as lignes,
       max(exporte_le) as exporte_le
from brulerie.fct_ventes
where cle_date >= '2025-06-11'
group by cle_date
order by cle_date;
-- Attendu : 2025-06-11, 47 tickets, 77 lignes, 2025-06-11 18:07:00 ;
-- 2025-06-12, 58, 94, 2025-06-12 15:42:00.
```

4.5 Livrer le 13 juin, puis faire passer le fait, en trois temps.

```
python 01_de\charger.py 2025-06-13
REM Attendu : rien ne s'affiche.
dbt run --select fct_ventes
REM Attendu : 1 of 1 OK created sql incremental model main.fct_ventes, puis Done. PASS=1.
```

Relancer la cellule de l'étape 4.4. Attendu : 2025-06-11, 47 tickets, 77 lignes, 2025-06-11 18:07:00 ; 2025-06-12, 64, 102, 2025-06-13 07:05:00 ; 2025-06-13, 54, 90, 2025-06-13 18:07:00.

Le 12 juin a ses 64 tickets payés et ses 102 lignes, tous de l'export de 7 h 05. Relancer aussi la cellule de l'étape 3.3 : elle ne rend plus aucune ligne.

Étape 5 — Le mois entier, et une passe relancée · Fiabiliser et faire tourner

5.1 Livrer le reste du mois, puis faire passer le fait, en trois temps. Le chargeur prend d'un coup les dix-sept jours qui restent. Une passe qui a pris du retard rattrape de même : elle prend tout ce qui est arrivé depuis la précédente.

© 2026 iAlexMG. Tous droits réservés.

```
python 01_de\charger.py 2025-06-30
REM Attendu : rien ne s'affiche ; la caisse porte de nouveau ses 31 exports, 1585 tickets et
REM 2626 lignes, comme au cours 00.
dbt run --select fct_ventes
REM Attendu : 1 of 1 OK created sql incremental model main.fct_ventes, puis Done. PASS=1.
```

Dans une cellule neuve :

```
-- 1. le fait entier : ses lignes, son montant, le plus récent export qu'il a rangé
select count(*) as lignes, sum(montant) as montant, max(exporte_le) as dernier_export
from brulerie.fct_ventes;
-- Attendu : 2509 lignes, 16718.00, le dernier export le 2025-06-30 18:07:00.
```

Le fait bâti par passes retombe sur celui du cours 04, bâti d'un bloc : 2 509 lignes et 16 718,00 $.

5.2 Relancer la passe sans rien livrer, en trois temps : `detach`, la commande, `attach`.

```
dbt run --select fct_ventes
REM Attendu : 1 of 1 OK created sql incremental model main.fct_ventes, puis Done. PASS=1.
```

Relancer la cellule de l'étape 5.1 : 2 509 lignes et 16 718,00 $, comme avant. Aucun export n'est plus récent que le dernier rangé, et la passe n'a rien pris. Une passe relancée après une panne, ou deux fois par erreur, ne change rien : on dit qu'elle est idempotente.

5.3 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

5.4 Tout lancer, dans la première fenêtre.

```
dbt build
REM Attendu : 43 of 43, puis Finished running 1 incremental model, 1 snapshot, 5 table models,
REM 29 data tests, 7 view models, et Done. PASS=43 WARN=0 ERROR=0.
```

Les tests passent sur le fait bâti par passes, le témoin ticket par ticket compris.

Si dbt s'arrête sur	**La cause, et le geste**
`IO Error: Cannot open file "…\brulerie.duckdb"`	le notebook tient l'entrepôt : relancer sa seconde cellule, le `detach`, puis la commande
`FAIL 6 assert_temoin_caisse` après l'étape 4	le fait n'a pas été refait après le changement de filtre : reprendre l'étape 4.4, puis la suite

© 2026 iAlexMG. Tous droits réservés.

Étape 6 — Versionner · Fiabiliser et faire tourner

6.1 Valider la vue des lignes, `int_ventes`, le fait et son réglage.

```
git -C 02_ae add models
git -C 02_ae commit -m "Cours 07 : le fait des ventes en incrémental"
REM Attendu : 4 files changed.
git -C 02_ae status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- Un fait incrémental ne traite, à chaque passe, que ce qui est arrivé depuis la précédente ; `--full-refresh` le refait de zéro, après la bascule et après tout changement de filtre.
- Le filtre compare l'instant d'arrivée, jamais le jour de l'événement : un export repris arrive après le jour qu'il décrit, et un filtre sur le jour de vente le perd sans erreur.
- `unique_key` fait remplacer une ligne reçue de nouveau au lieu de la doubler, et rend la passe idempotente ; seul un témoin comparé à la source voit une ligne perdue.

© 2026 iAlexMG. Tous droits réservés.